

Cloud Infrastructure Monitoring & Auto-Deployment System

A GCP-based monitoring, alerting, and CI/CD deployment platform

Project Report

Prepared by: Parth Gulbhile

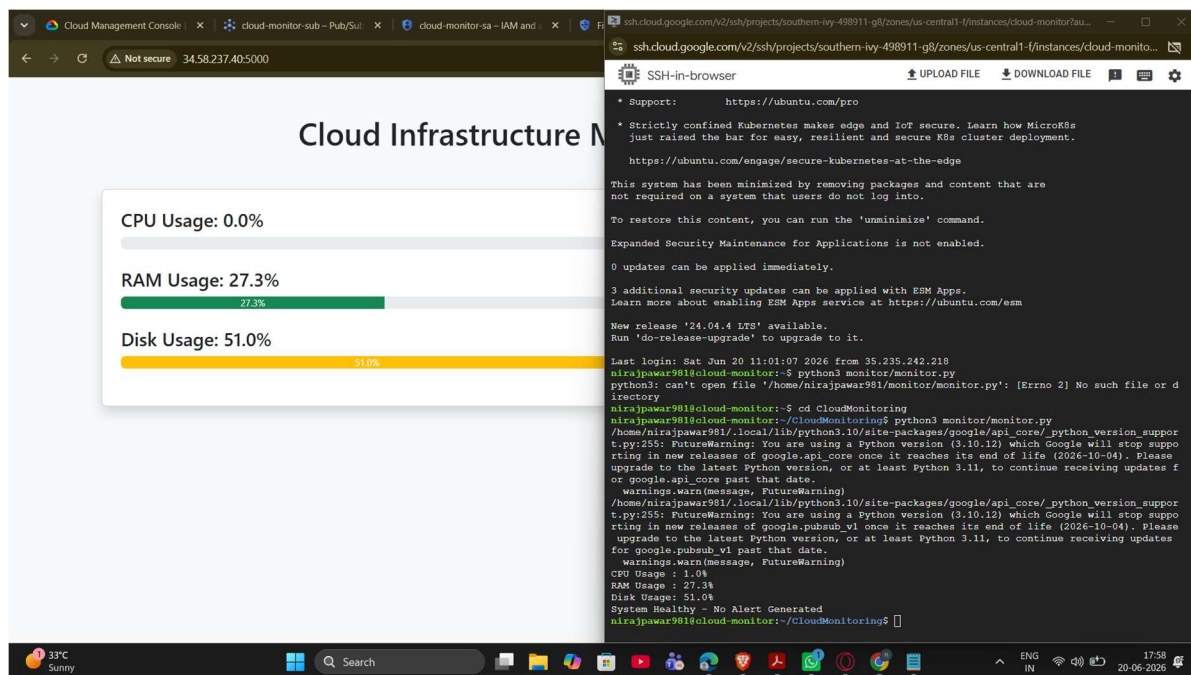
Niraj Pawar

Roll Number: 24108B0057

24108B0078

Repository: github.com/Parth-2006/CloudMonitoring

Platform: Google Cloud Platform · Ubuntu 22.04 LTS



Final dashboard rendering live metrics, served from inside a Docker container on the GCP VM.

Table of Contents

Table of Contents.....	2
1. Objective.....	3
2. Technology Stack	3
3. System Architecture	3
Monitoring Layer	3
Visualization Layer.....	3
Alerting Layer.....	3
Deployment Layer.....	4
4. Implementation Journey	5
4.1 Cloud Platform Selection	5
4.2 VM Environment Setup.....	5
4.3 Monitoring Script (monitor.py).....	5
4.4 Flask Dashboard — First Version	5
Debugging the 500 Internal Server Error.....	6
Other Issues Hit Along the Way	7
4.5 Automation with Cron	8
4.6 Bootstrap Dashboard UI	8
4.7 Docker Containerization	9
4.8 Google Cloud Pub/Sub Alerting	9
IAM & Service Account Configuration	10
4.9 Networking & Firewall	11
4.10 GitHub Actions CI/CD	12
4.11 Final Verification	12
5. Major Errors Encountered & Resolved	13
6. Outcome	14
7. Key Learning Outcomes.....	14
8. Planned Next Steps	14
9. Repository Structure	15

1. Objective

To design and implement a cloud-native monitoring and deployment platform that:

- Continuously monitors VM resource utilization (CPU, RAM, Disk)
- Displays metrics through a live, auto-refreshing web dashboard
- Publishes threshold-based alerts via Google Cloud Pub/Sub
- Automatically deploys application updates using GitHub Actions and Docker

2. Technology Stack

Category	Technologies
Cloud Platform	Google Cloud Platform (Compute Engine)
Operating System	Ubuntu 22.04 LTS
Languages	Python, Bash
Web Framework	Flask
Monitoring	psutil
Frontend	Bootstrap
Containerization	Docker
Messaging	Google Cloud Pub/Sub
Automation	Cron Jobs
Version Control	Git & GitHub
CI/CD	GitHub Actions
Data Format	JSON

3. System Architecture

The system is organized into four logical layers:

Monitoring Layer

monitor.py collects CPU, RAM, and Disk utilization using psutil and writes the results to data.json.

Visualization Layer

A Flask dashboard reads data.json and renders the metrics using Bootstrap progress bars, auto-refreshing every 5 seconds.

Alerting Layer

monitor.py checks the collected metrics against configured thresholds. If a threshold is exceeded, an alert message is published to a Google Cloud Pub/Sub topic, where a subscriber receives and displays it.

Deployment Layer

GitHub Actions automates deployment: every push to the main branch triggers an SSH connection into the VM, a git pull, a Docker image rebuild, and a container restart.

```
Git Push -> GitHub Actions -> SSH into VM -> Git Pull -> Docker Rebuild -> Container Restart
```

4. Implementation Journey

4.1 Cloud Platform Selection

The project initially started on AWS EC2 using Amazon Linux, but ran into package-management friction: Ubuntu-style “apt” commands did not work on Amazon Linux (which uses “yum”/“dnf”), pip3 and the psutil package were missing, and environment setup became inconsistent across versions.

The project was migrated to Google Cloud Platform on an Ubuntu 22.04 LTS, e2-micro Compute Engine instance for smoother compatibility with course tooling and easier package installation.

4.2 VM Environment Setup

A Compute Engine VM was provisioned with HTTP/HTTPS access enabled. SSH access was configured, and the system was updated. Python3, pip, Git, Docker, and Nginx were installed.

Challenge	Resolution
Docker permission errors	Added the user to the docker group
Package / dependency issues	Verified installs manually, one by one

4.3 Monitoring Script (monitor.py)

Built using psutil.cpu_percent(), psutil.virtual_memory(), and psutil.disk_usage(), writing output to monitor/data.json:

```
{  "cpu": 50.0,  "ram": 26.1,  "disk": 48.3 }
```

Challenge	Resolution
ModuleNotFoundError: No module named 'psutil'	Installed the package using pip
File path / location mismatches	Standardized project structure; used absolute paths

4.4 Flask Dashboard — First Version

A minimal Flask app was built to read data.json and render plain HTML.

```
nirajpawar981@cloud-monitor:~/CloudMonitoring/dashboard$ cat ~/CloudMonitoring/dashboard/app.py
from flask import Flask
import json

app = Flask(__name__)

@app.route("/")
def home():

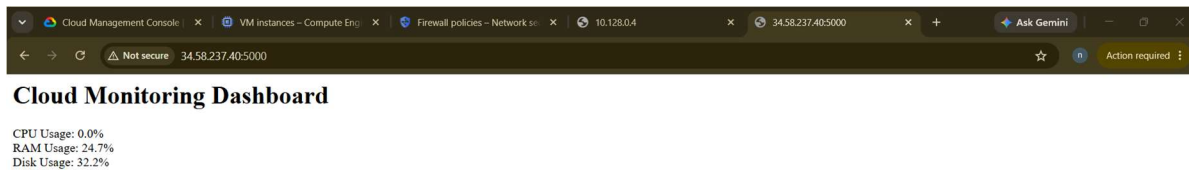
    with open("../monitor/data.json") as f:
        data = json.load(f)

    return f"""
    <h1>Cloud Monitoring Dashboard</h1>

    CPU Usage: {data['cpu']}%<br>
    RAM Usage: {data['ram']}%<br>
    Disk Usage: {data['disk']}%
    """

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
nirajpawar981@cloud-monitor:~/CloudMonitoring/dashboard$
```

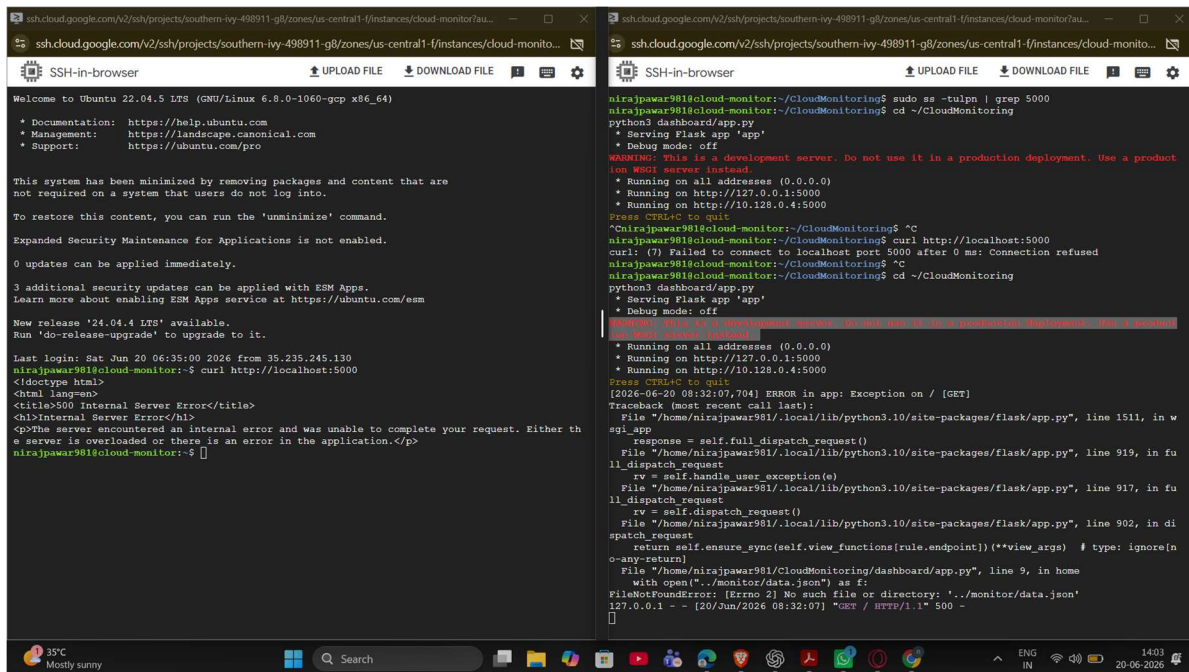
Early version of app.py — note the relative path “../monitor/data.json”, which later caused a 500 error.



First successful dashboard render, accessed via the VM's public IP on port 5000.

Debugging the 500 Internal Server Error

Running the Flask app from a different working directory broke the relative path to data.json, throwing a FileNotFoundError wrapped inside a 500 response.



Two SSH-in-browser panes side by side: left shows `curl localhost:5000` returning a 500 page; right shows the full Flask traceback — `FileNotFoundError: [Errno 2] No such file or directory: '../monitor/data.json'`.

Root cause: relative path resolution depends on the current working directory the process was launched from, not the script's own location.

Fix: switched to dynamic, absolute path resolution using `os.path.join()` and `os.path.dirname(__file__)`.

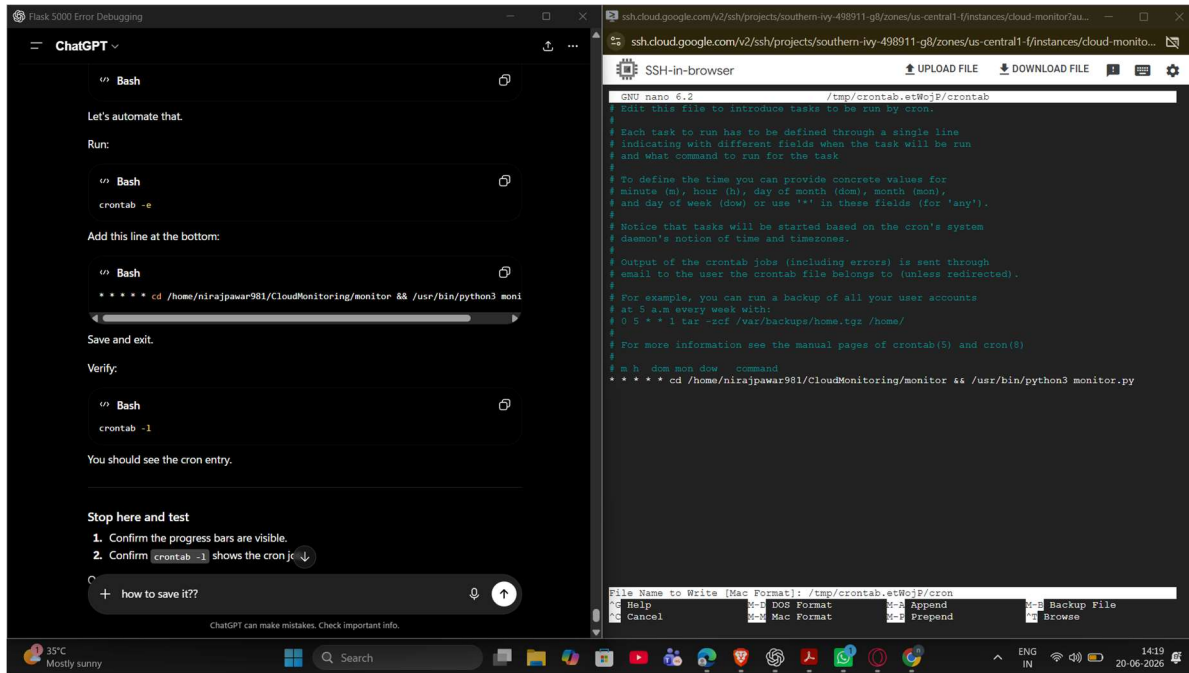
Other Issues Hit Along the Way

Issue	Cause	Fix
Address already in use (port 5000)	A Docker container was already bound to port 5000	Found the culprit with “ <code>sudo ss -tulpn grep 5000</code> ” and stopped it
python: command not found	Ubuntu ships Python3 only	Used python3 explicitly
SyntaxError on Flask import	Typo in import statement	Corrected “from flask import Flask”

4.5 Automation with Cron

A cron job was configured to run `monitor.py` every minute so `data.json` stays fresh without manual intervention:

```
* * * * * cd /home/nirajpawar981/CloudMonitoring/monitor && /usr/bin/python3 monitor.py >> cron.log 2>&1
```

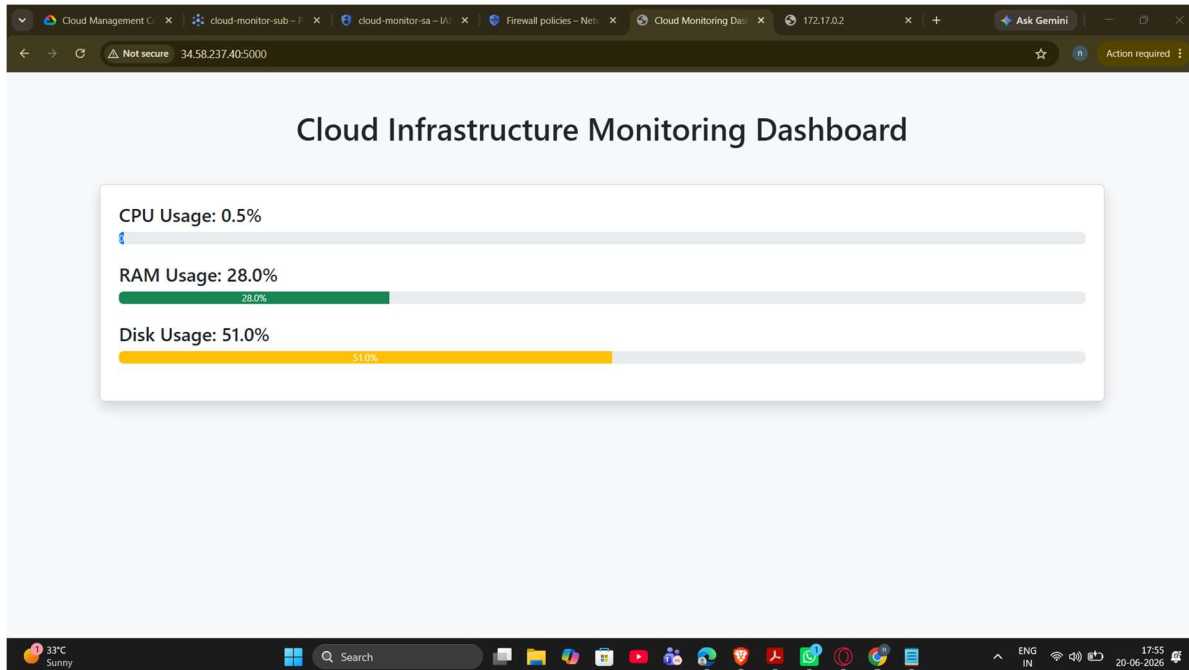


Editing the crontab through “`crontab -e`” over SSH-in-browser. Logging output to `cron.log` was added after the dashboard appeared to stop updating, to confirm the job was actually firing.

Challenge	Resolution
“new crontab file is missing newline before EOF”	Added a trailing newline; later used direct command-based cron install
Dashboard values appeared frozen	Cron failures were silent — added logging to surface errors and confirm successful runs

4.6 Bootstrap Dashboard UI

The plain-HTML dashboard was upgraded to a Bootstrap-based UI with colour-coded progress bars and 5-second auto-refresh.



Upgraded dashboard — CPU, RAM, and Disk shown as colour-coded Bootstrap progress bars, refreshing automatically.

4.7 Docker Containerization

A Dockerfile was written, the image was built, and the Flask dashboard was run inside a container (cloud-monitor-container), verified with docker ps.

A subtler bug surfaced here: the Dockerfile's COPY instruction only copies data.json at build time, so the container's copy diverged from the host's copy as cron kept updating the host file.

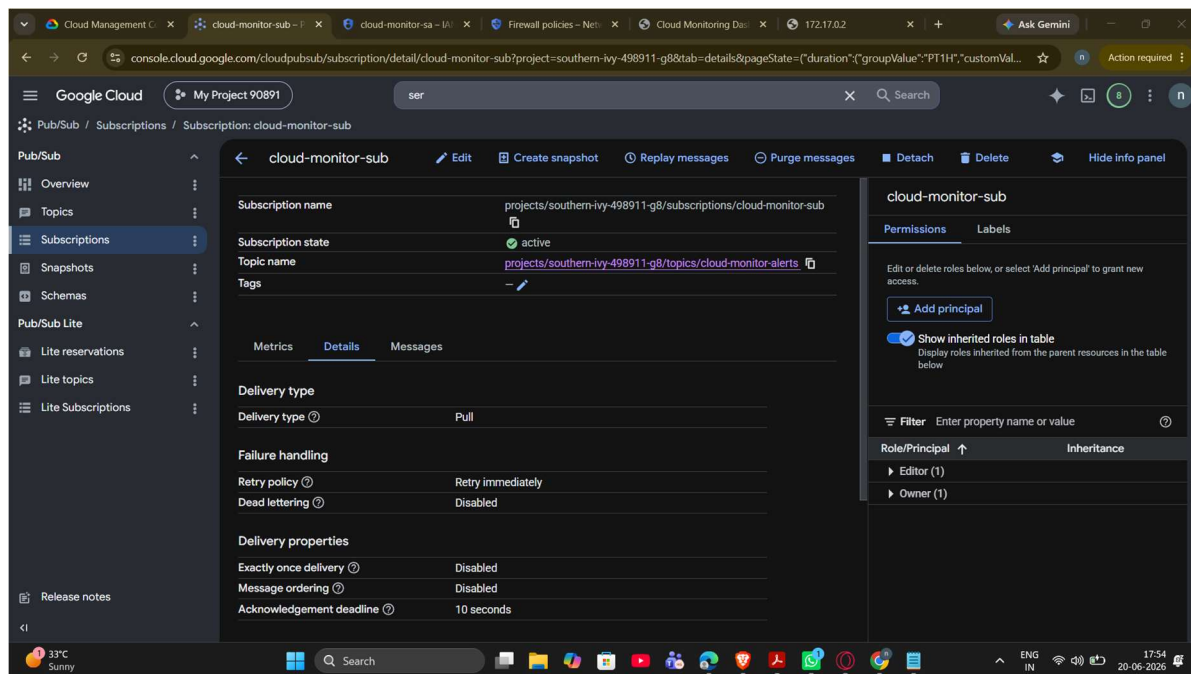
Fix: replaced the static copy with a Docker volume mount:

```
-v /home/nirajpawar981/CloudMonitoring/monitor:/app/monitor
```

This bind-mounts the host's monitor/ directory directly into the container, so every cron-driven update is reflected in the dashboard instantly — no rebuild required.

4.8 Google Cloud Pub/Sub Alerting

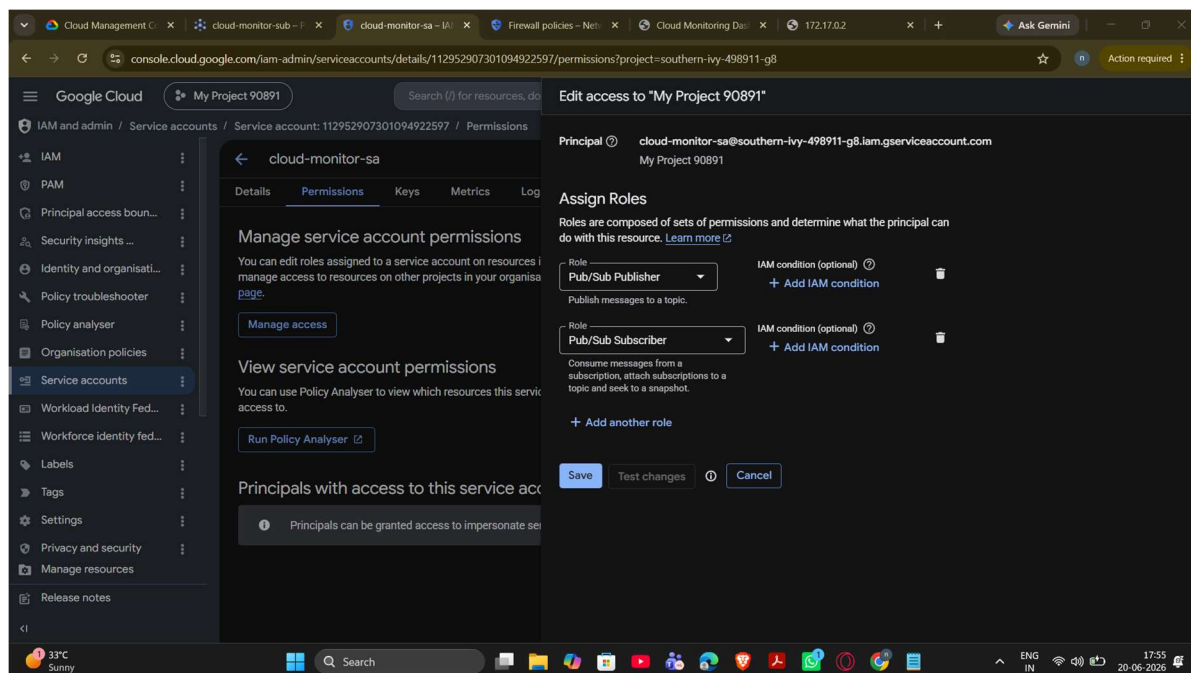
A Pub/Sub topic (cloud-monitor-alerts) and subscription (cloud-monitor-sub) were set up so monitor.py can publish an alert whenever a threshold is breached, to be picked up by a subscriber process.



cloud-monitor-sub — a Pull-type subscription attached to the cloud-monitor-alerts topic, shown active in the GCP console.

IAM & Service Account Configuration

A dedicated service account (*cloud-monitor-sa*) was created and granted the Pub/Sub Publisher and Pub/Sub Subscriber IAM roles so *monitor.py* could authenticate and publish/consume messages without using a personal account.



IAM role assignment for cloud-monitor-sa — Publisher and Subscriber roles granted at the project level.

Challenge	Resolution
Permission Denied / insufficient OAuth scope	Granted explicit Publisher/Subscriber roles to the service account
Credential path issues	Set GOOGLE_APPLICATION_CREDENTIALS to point at the downloaded service account key

4.9 Networking & Firewall

Port 5000 (used by Flask/Docker) was not reachable externally by default on GCP, so a custom VPC firewall rule was created to allow ingress traffic on tcp:5000.

The screenshot shows the Google Cloud console interface for Firewall policies. The left sidebar lists various network security options, with 'Firewall policies' selected. The main panel displays a table of firewall rules. A custom rule named 'allow-5000' is highlighted, showing it allows TCP traffic on port 5000 from all IP ranges. Below the table, a notification states 'Successfully created firewall rule allow-5000.'

Name	Type	Targets	Filters	Protocols/ports	Action	Priority	Network	Logs
allow-5000	Ingress	Apply to all	IP ranges:	tcp:5000	Allow	1000	default	Off
default-allow-http	Ingress	http-server	IP ranges:	tcp:80	Allow	1000	default	Off
default-allow-https	Ingress	https-server	IP ranges:	tcp:443	Allow	1000	default	Off
default-allow-icmp	Ingress	Apply to all	IP ranges:	icmp	Allow	65534	default	Off
default-allow-internal	Ingress	Apply to all	IP ranges:	tcp:0-65535 udp:0-65535 icmp	Allow	65534	default	Off
default-allow-rdp	Ingress	Apply to all	IP ranges:	tcp:3389	Allow	65534	default	Off
default-allow-ssh	Ingress	Apply to all	IP ranges:	tcp:22	Allow	65534	default	Off

Custom ingress firewall rule “allow-5000” permitting tcp:5000 from all IP ranges, alongside GCP’s default rules (SSH, RDP, HTTP/HTTPS, ICMP).

4.10 GitHub Actions CI/CD

The deployment pipeline was automated so every push to main triggers:

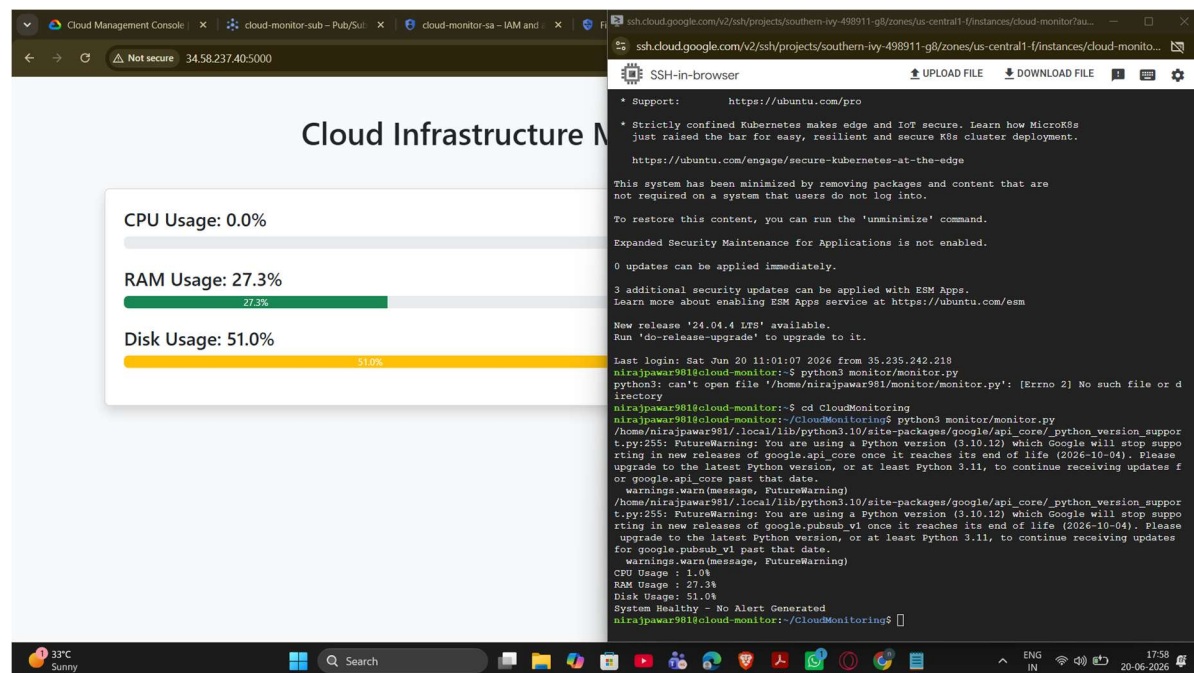
Developer Push -> GitHub Repository -> GitHub Actions -> SSH into GCP VM -> Git Pull -> Docker Rebuild -> Container Restart

Challenge	Resolution
ssh.ParsePrivateKey: ssh: no key found	Regenerated SSH key pair; added private key to GitHub Secrets, public key to authorized_keys
sudo: a password is required	Removed unneeded sudo calls — the deploy user was already in the docker group
YAML indentation errors in deploy.yml	Carefully corrected spacing per YAML syntax rules

GitHub Secrets configured: VM_HOST, VM_USER, VM_SSH_KEY.

4.11 Final Verification

An end-to-end test confirmed the monitoring script runs cleanly, writes fresh metrics, and the dashboard reflects them live through the Docker volume mount.



Right pane: “python3 monitor/monitor.py” executes successfully (CPU 1.0%, RAM 27.3%, Disk 51.0%, “System Healthy — No Alert Generated”). Left pane: the same values rendered live on the Bootstrap dashboard.

5. Major Errors Encountered & Resolved

#	Error	Root Cause → Fix
1	FileNotFoundError reading data.json	Relative path broke when run from a different working directory → absolute path resolution via os.path
2	Flask 500 Internal Server Error	Same root cause as above, surfaced as an unhandled exception → same fix, plus better error logging
3	Docker container showing stale metrics	COPY only runs at image build time → switched to a Docker volume mount
4	Address already in use (port 5000)	An existing Docker container already bound the port → identified via ss -tulpn, stopped it
5	GitHub Actions SSH failure	Malformed / missing private key in GitHub Secrets → regenerated key pair, reconfigured secrets
6	GitHub Actions YAML errors	Indentation mistakes in deploy.yml → corrected YAML spacing
7	Pub/Sub Permission Denied / scope errors	Service account lacked Publisher/Subscriber roles → granted IAM roles explicitly
8	Cron job not updating data.json	Cron failures were silent → added logging to confirm execution
9	"new crontab file is missing newline before EOF"	Crontab file lacked a trailing newline → added newline, reinstalled crontab
10	python: command not found	Ubuntu ships only python3 → used python3 explicitly

6. Outcome

The completed system:

- Continuously monitors CPU, RAM, and Disk usage on a GCP VM
- Displays live metrics on an auto-refreshing Bootstrap dashboard
- Publishes and consumes threshold alerts through Google Cloud Pub/Sub
- Runs inside a Docker container, kept in sync with the host via a volume mount
- Redeploys automatically on every push via a GitHub Actions CI/CD pipeline
- Is reachable externally through a custom GCP firewall rule on port 5000

This project demonstrates practical, hands-on experience across Linux administration, GCP VM management, Python scripting, monitoring automation, Docker containerization, CI/CD pipelines, event-driven architecture (Pub/Sub), and general DevOps debugging.

7. Key Learning Outcomes

- Linux server administration and process debugging
- Cloud VM provisioning and networking (GCP Compute Engine, VPC firewalls)
- Building monitoring tools with Python and psutil
- Task scheduling and automation with cron
- Git/GitHub version control and CI/CD with GitHub Actions
- Flask web application development
- Docker containerization, including the host-vs-container data sync pitfall
- Event-driven design using Google Cloud Pub/Sub
- Systematic debugging using logs, tracebacks, and terminal tools

8. Planned Next Steps

- Docker Compose for multi-container orchestration
- Telegram / email alert notifications on threshold breach
- HTTPS termination via an Nginx reverse proxy
- Persistent metrics history (time-series storage) instead of a single-point data.json
- Unit tests for monitor.py and the Flask routes

9. Repository Structure

```

CloudMonitoring/ └─ monitor/ │   └─ monitor.py           # Collects CPU/RAM/Disk metrics │
└─ data.json          # Latest metrics snapshot └─ dashboard/ │   └─ app.py           #
Flask app serving the dashboard │   └─ templates/        # Bootstrap UI templates └─
Dockerfile └─ .github/ │   └─ workflows/ │   └─ deploy.yml  # CI/CD pipeline
definition └─ README.md

```

Report compiled from project logs, terminal sessions, and console screenshots captured during development on GCP.